

Connections Between SCX Self-Evolution and Known Theories

Part of the SCX Self-Evolution Theory Series Status: Formal comparison | **Audit:** Pre-review **Prerequisites:** THEOREMS_UNIFIED.md, self-evolution definitions (Files 1-7)

Table of Contents

1. AlphaZero Self-Play
 2. Bayesian Optimization
 3. Active Learning
 4. Solomonoff Induction
 5. Comprehensive Comparison Table
 6. Synthesis: What Is and Is Not New
-

1. AlphaZero Self-Play

1.1 Overview

AlphaZero (Silver et al., 2017, 2018) learns to play board games through self-play reinforcement learning. The agent plays games against itself, using Monte Carlo Tree Search (MCTS) to generate training data, which is then used to update a deep neural network. The network has two heads: a policy

head $p(s)$ predicting move probabilities and a value head $V(s)$ predicting the expected outcome from state s .

The self-play loop is:

$$\pi_{\theta_{\text{old}}} \xrightarrow{\text{MCTS}} \{\text{self-play games}\} \xrightarrow{\text{training}} \pi_{\theta} \xrightarrow{\text{evaluation}} \pi_{\theta_{\text{new}}}$$

1.2 Formal Mapping

The SCX self-evolution loop maps to AlphaZero as follows:

AlphaZero Component	SCX Self-Evolution Component	Rationale
Policy network π_{θ}	Gatekeeper S_t	Both select actions: π_{θ} selects moves, S_t selects data for validation
Self-play games	NEP student f_{θ_t} predictions	Both generate training targets through interaction with a fixed environment

CONNECTIONS BETWEEN SCX SELF-EVOLUTION AND KNOWN THEORIES³

AlphaZero Component	SCX Self-Evolution Component	Rationale
MCTS search	SCX state-conditioned aggregation $\hat{R}_m(s), \text{SCX}_m(s)$	Both refine raw evaluations through structured computation
Value network $V(s)$	SCX state value function $V(s) = \mathbb{E}[\text{improvement} \mid s]$	Both estimate the long-term value of states/decisions
Replay buffer	Memory bank M_t	Both store experience for future training
Evaluation against previous version	Lyapunov descent check $\Phi(S_t) < \Phi(S_{t-1})$	Both compare new policy against old to ensure monotonic improvement

AlphaZero Component	SCX Self-Evolution Component	Rationale
Opponent (fixed rules)	True physical law f^* + current data $\mathcal{D}_t$	Both provide the fixed, non- learnable environment
Training loss $L(\theta) = L_{\text{policy}} + L_{\text{value}} + L_{\text{reg}}$	Lyapunov function $\Phi(S_t, M_t, f_{\theta_t})$	Both provide the scalar signal for optimization

1.3 Key Similarities

1. **Bootstrapping:** Both systems generate their own training data. AlphaZero’s training data comes from MCTS self-play; SCX’s training data comes from the NEP student’s self-generated configurations with gatekeeper-selected validation.
2. **Monotonic improvement:** AlphaZero uses policy iteration, which guarantees non-decreasing performance against the previous version. SCX uses Lyapunov descent, which guarantees $\Phi(S_t) \leq \Phi(S_{t-1})$.
3. **Memory buffer:** Both store past experience. AlphaZero’s replay buffer prevents catastrophic forgetting; SCX’s memory bank M_t preserves previously validated data.
4. **Exploration-exploitation:** AlphaZero’s MCTS balances exploration (via Dirichlet noise in prior probabilities) with exploitation (via value-

guided search). SCX’s gatekeeper balances exploration (validating uncertain states) with exploitation (trusting states with high SCX reliability scores).

1.4 Key Differences

1. **Data generation mechanism:** AlphaZero generates **synthetic** games through self-play. The environment (game rules) is completely known and simulable. SCX generates data through NEP **simulations** of physical systems. The environment (physical law f^*) is **unknown** and only partially observable through noisy labels.
2. **Feedback delay:** AlphaZero receives immediate game outcomes (win/loss/draw) at the end of each game. The value target is unambiguous. SCX’s feedback from NEP validation is **delayed** and **partial**: only a subset of samples are validated, and the validation itself is noisy (DFT approximations, experimental error).
3. **Convergence mechanism:** AlphaZero uses **policy iteration** — solving a series of increasingly accurate supervised learning problems against a fixed opponent. SCX uses **memory-augmented Bayesian update** — the gatekeeper is updated based on accumulated evidence in M_t , and the NEP student is updated based on the full memory bank.
4. **Distribution shift:** AlphaZero’s data distribution shifts as the policy improves, but the game dynamics remain fixed. SCX’s distribution shifts **doubly**: the gatekeeper changes which data are selected for validation, and the NEP student changes the space of accessible configurations.

5. **Final state:** AlphaZero converges to a superhuman policy (in principle, the Nash equilibrium of the game if the network has sufficient capacity). SCX converges to a self-consistent fixed point (Theorem SE-2) that may be **suboptimal** due to the finite configuration space and the fundamental unidentifiability of noise (Theorem 3).

1.5 Convergence Comparison

AlphaZero (policy iteration):

$$\text{Performance}(\pi_{t+1}) \geq \text{Performance}(\pi_t)$$

Convergence to optimal policy π^* under sufficient capacity and exploration (proven for tabular case; empirically observed for neural networks).

SCX (Lyapunov descent):

$$\Phi(S_{t+1}, M_{t+1}, f_{\theta_{t+1}}) \leq \Phi(S_t, M_t, f_{\theta_t})$$

Convergence to fixed point q_{T^*} in finite time (Theorem SE-2). The fixed point is self-consistent but not necessarily optimal in a global sense.

Comparison: Both guarantee monotonic improvement. AlphaZero's guarantee is stronger (convergence to optimal play under certain conditions) but relies on a fully known environment. SCX's guarantee is weaker (convergence to a self-consistent fixed point) but operates in an environment with unknown ground truth.

2. Bayesian Optimization

2.1 Overview

Bayesian optimization (BO) (Mockus, 1975; Jones et al., 1998) optimizes a black-box function $f : \mathcal{X} \rightarrow \mathbb{R}$ by building a probabilistic surrogate model g (typically a Gaussian process) and using an acquisition function α to select the next evaluation point:

$$x_{t+1} = \arg \max_{x \in \mathcal{X}} \alpha(x \mid g_{1:t})$$

where α balances exploration (points with high uncertainty) and exploitation (points with high predicted value).

2.2 Formal Mapping

BO Component	SCX Self-Evolution Component	Rationale
Black-box function f	True physical law f^*	Both are unknown, expensive to evaluate
Surrogate model g	NEP student f_θ	Both approximate the unknown ground truth

CONNECTIONS BETWEEN SCX SELF-EVOLUTION AND KNOWN THEORIES8

BO Component	SCX Self-Evolution Component	Rationale
Acquisition function α	Gatekeeper S_t	Both guide the selection of the next evaluation point
Evaluation budget	NEP validation budget	Both have limited evaluations (BO: function evaluations; SCX: DFT/experiment budget)
Posterior $p(f \mid \mathcal{D}_{1:t})$	Memory bank M_t + student f_{θ_t}	Both represent accumulated knowledge about the unknown function
Regret $r_t =$ $f(x^*) - f(x_t)$	Lyapunov gap $\Phi(q_t) - \Phi_{\text{opt}}$	Both measure suboptimality of the current state

2.3 Formal Comparison: Acquisition Functions

BO's Expected Improvement (EI):

$$\alpha_{\text{EI}}(x) = \mathbb{E}[\max(0, f(x) - f_{\text{best}}) \mid \mathcal{D}_{1:t}]$$

SCX's State Data Value:

$$V(s) = \underbrace{\hat{R}(s)}_{\text{expert disagreement}} \cdot \underbrace{\rho(s)}_{\text{state proportion}} \cdot \underbrace{(1 - C(s))}_{\text{uncertainty}}$$

where $C(s) = \frac{1}{M} \sum_m \mathbf{1}\{\ell(f_m(x), y) > \tau\}$ is the consensus score (Theorem 1).

Both EI and SCX's $V(s)$ balance: - **Exploitation** (high predicted improvement / high expert disagreement $\hat{R}(s)$) - **Exploration** (high posterior uncertainty / low consensus $C(s)$)

Key difference: EI operates on a continuous input space and is formalized as an expectation under a Gaussian process posterior. SCX's $V(s)$ operates on a discrete state space (partition of $\mathcal{X}$ into K_S states) and uses frequentist estimates of within-state disagreement.

2.4 Convergence Rates

BO convergence (Srinivas et al., 2010): For a GP with kernel k , the cumulative regret is bounded by:

$$R_T = \sum_{t=1}^T r_t \leq \sqrt{T \cdot \gamma_T \cdot C}$$

where γ_T is the maximum information gain after T evaluations. For the squared exponential kernel, $\gamma_T = O((\log T)^{d+1})$, giving $R_T = O(\sqrt{T(\log T)^{d+1}})$.

SCX convergence (Theorem SE-1, SE-2): For the Lyapunov function Φ , the gap decays as:

$$\Phi(q_t) - \Phi_{\text{opt}} \leq \Phi(q_0) \cdot \exp(-\lambda t)$$

under the contraction condition, or in worst case:

$$T^* \leq \frac{\Phi_0}{\varepsilon_{\text{mach}}}$$

Comparison: BO provides cumulative regret bounds that depend on the information-theoretic complexity of the function class. SCX provides a finite-time termination bound that depends on the Lyapunov descent rate and machine precision. BO’s bounds are more informative for sub-exponential convergence; SCX’s guarantee is stronger (exact termination).

2.5 Key Similarities

1. **Uncertainty-guided exploration:** Both systems use uncertainty estimates to allocate evaluation resources where they are most needed.
2. **Surrogate-based decision making:** Both maintain an internal model (GP / NEP student) that guides data acquisition.
3. **Balanced exploration-exploitation:** Both use explicit mechanisms to balance exploring unknown regions against exploiting known good regions.

4. **Sequential design:** Both are iterative, adding one (or a batch) of evaluation points per iteration.

2.6 Key Differences

1. **Target distribution:** BO optimizes a **static** black-box function. The objective f is fixed across iterations. SCX faces an **evolving** target distribution: as the NEP student improves, the set of “interesting” configurations changes.
 2. **Batch selection:** SCX typically validates an entire batch of samples (all samples in a selected state) before updating the gatekeeper. BO typically evaluates one point at a time (though batch BO variants exist).
 3. **State structure:** SCX leverages the state partition S to aggregate information across similar inputs. BO treats each input point independently (though GP kernels induce similarity structure).
 4. **Multiple objectives:** SCX’s gatekeeper manages multiple criteria (noise detection, expert disagreement, data diversity). BO typically optimizes a single scalar objective (though multi-objective BO variants exist).
 5. **Cost structure:** BO typically assumes homogeneous evaluation costs. SCX’s validation cost varies across states (DFT calculations for some configurations are more expensive than others).
-

3. Active Learning

3.1 Overview

Active learning (AL) (Settles, 2009) aims to reduce labeling effort by selectively querying labels for the most informative unlabeled examples. Given a labeled pool $\mathcal{L}$, an unlabeled pool $\mathcal{U}$, and a model h , the query strategy $Q(x \mid h)$ selects the next sample(s) to label.

3.2 Formal Mapping

AL Component	SCX Self-Evolution Component	Rationale
Model h	NEP student f_θ	Both are the predictive model being improved
Oracle O (labeler)	NEP simulator + DFT validation	Both provide ground-truth labels
Query strategy $Q(x \mid h)$	Gatekeeper S_t + state-value scoring	Both select which samples to label/validate
Labeled pool $\mathcal{L}$	Memory bank M_t	Both store labeled/validated data

AL Component	SCX Self-Evolution Component	Rationale
Unlabeled pool $\mathcal{U}$	Candidate pool $\mathcal{C}_t$	Both contain unlabeled samples
Labeling budget	NEP validation budget	Both have finite labeling capacity

3.3 Comparison of Query Strategies

Standard AL strategies:

1. **Uncertainty sampling:** $Q(x) = 1 - \max_y P(y | x)$, selects samples where the model is most uncertain.
2. **Query-by-committee (QBC):** $Q(x) = \frac{1}{M} \sum_{m=1}^M \mathbf{1}\{h_m(x) \neq \bar{h}(x)\}$, selects samples with maximum disagreement among an ensemble of models.
3. **Expected model change:** $Q(x) = \mathbb{E}_{y \sim P(y|x)} [\|\nabla L(\theta \cup (x, y)) - \nabla L(\theta)\|]$, selects samples expected to maximally change the model.

SCX's approach (state-certified active learning):

$$a^*(s) = \arg \max_{a \in \mathcal{A}} U(a, s)$$

where $\mathcal{A} = \{\text{validate, discard, defer, ...}\}$ and $U(a, s)$ is the expected utility of action a for state s . The utility is computed from state-level statistics:

$$U(\text{validate}, s) = \underbrace{\hat{R}(s)}_{\text{expert disagreement}} \cdot \underbrace{(1 - \text{SCX}(s))}_{\text{reliability gap}} \cdot \underbrace{\rho(s)}_{\text{state size}}$$

3.4 Comparison of Query Strategies

Strategy	AL Counterpart	SCX Version	Key Difference
Uncertainty sampling	$1 - \max_y P(y \mid x)$	$1 - C(s)$ where $C(s)$ is state consensus	SCX uses multi-expert consensus instead of single-model uncertainty
Query-by-committee	Vote entropy among ensemble	SCX reliability gap $\text{SCX}(s)$	SCX uses state-conditioned expert reliability, not raw vote entropy
Expected model change	$\ \nabla L(\theta \cup (x, y)) - \nabla L(\theta)\ $	$V(s) = \hat{R}(s) \cdot \rho(s) \cdot (1 - C(s))$	SCX's value function operates at state level, not point level
Density-weighted Diversity sampling	$\frac{1}{n} \sum_{i=1}^n \text{sim}(x, x_i)$ Coreset selection	$\rho(s)$ (state proportion) Discard redundant states via SCX-Compress	Both weight by representativeness Both aim for coverage

3.5 SCX’s State-Certified Active Learning

The key innovation of SCX’s approach is the **state-level certification**. Unlike point-wise active learning, which treats each sample independently:

1. **State aggregation:** SCX estimates statistics at the state level, pooling information across all samples in state s . This provides more robust estimates when individual labels are noisy.
2. **Certified actions:** Each action $a \in \mathcal{A}$ is certified by state-level guarantees. For example, “validate state s ” is justified when $\text{SCX}(s)$ is below threshold, indicating that the multi-expert consensus is unreliable for this state.
3. **Cascading decisions:** SCX makes decisions hierarchically: first at the state level (which state to focus on), then at the sample level (which samples within the state to validate).

3.6 Sample Complexity Comparison

Standard AL sample complexity (Balcan et al., 2009): Under the disagreement coefficient θ , active learning achieves label complexity:

$$N_{\text{AL}}(\varepsilon) = O\left(\theta \cdot d \cdot \log \frac{1}{\varepsilon}\right)$$

where d is the VC dimension of the hypothesis class.

SCX sample complexity (derived from Theorem 1 and Proposition 2): The number of validation labels needed per state s to achieve reliability $\text{SCX}_m(s) > 1 - \delta$ is:

$$n_{\text{SCX}}(s, \delta) = O\left(\frac{1}{\Delta_s^2} \log \frac{1}{\delta}\right)$$

where Δ_s is the state-level separation gap (Theorem 1). Aggregating over K_S states:

$$N_{\text{SCX}}(\varepsilon) = O\left(K_S \cdot \frac{1}{\Delta_{\min}^2} \log \frac{1}{\varepsilon}\right)$$

where $\Delta_{\min} = \min_s \Delta_s$.

Comparison: Standard AL avoids the K_S multiplicative factor but requires the disagreement coefficient θ to be small. SCX’s factor of K_S is the cost of operating without a single discriminative model — SCX uses multiple experts and must characterize each state independently.

4. Solomonoff Induction

4.1 Overview

Solomonoff induction (Solomonoff, 1964) is a theoretical framework for inductive inference. Given a sequence of observations, the posterior probability of a continuation is:

$$P(x_{n+1} \mid x_1, \dots, x_n) = \frac{\sum_{p: p(x_1, \dots, x_n) = x_1, \dots, x_n} 2^{-|p|} \cdot P(x_{n+1} \mid p)}{\sum_{p: p(x_1, \dots, x_n) = x_1, \dots, x_n} 2^{-|p|}}$$

where p ranges over all programs for a universal Turing machine, $|p|$ is program length, and $P(x_{n+1} \mid p)$ is the continuation probability under program

p . The universal prior $M(x) = \sum_p 2^{-|p|} \cdot \mathbf{1}\{p(\varepsilon) = x\}$ assigns higher probability to strings generated by shorter programs (Occam’s razor).

4.2 Formal Analogy

Claim (SE-A1): The limiting SCX gatekeeper $S_\infty(x) = \lim_{t \rightarrow \infty} S_t(x)$ approximates a Solomonoff predictor for the question “Is this label correct?”

The analogy proceeds as follows:

Solomonoff	SCX Self-Evolution	Rationale
Universal prior $M(x) = \sum_p 2^{- p } \mathbf{1}\{p(\varepsilon) = x\}$	State partition $\mathcal{S}$ with prior $\rho_s = \mathbb{P}(X \in s)$	Both define prior beliefs over hypotheses
Programs p	Expert models $\{f_m\}$	Both are candidate explanations of observed data
Program length $ p $ (Occam penalty)	Expert complexity (implicit in training data requirement)	Both penalize complexity
Likelihood $P(\text{data} \mid p)$	Consensus score $C(x) = \frac{1}{M} \sum_m \mathbf{1}\{\ell(f_m(x), y) > \tau\}$	Both measure how well the hypothesis explains the observation
Posterior $P(x_{n+1} \mid \text{data})$	Gatekeeper score $S_t(x)$	Both represent the current belief about an unseen property

Solomonoff	SCX Self-Evolution	Rationale
Accumulated evidence	Memory bank M_t	Both store the history of observations
Convergence as $n \rightarrow \infty$	Convergence as $t \rightarrow \infty$ (Theorem SE-2)	Both converge to fixed points given sufficient evidence

4.3 Formal Correspondence

The Solomonoff posterior for the proposition “ x is clean” (i.e., $y = f^*(x)$) is:

$$P(\text{clean} \mid \mathcal{D}) = \frac{\sum_p 2^{-|p|} \cdot P(\mathcal{D} \mid p) \cdot \mathbf{1}\{p \text{ predicts clean at } x\}}{\sum_p 2^{-|p|} \cdot P(\mathcal{D} \mid p)}$$

The SCX gatekeeper’s score for x at state $s(x)$ is:

$$S_t(x) = \frac{1}{M} \sum_{m=1}^M \underbrace{\text{SCX}_m(s(x))}_{\text{expert reliability in state } s} \cdot \underbrace{\mathbf{1}\{\ell(f_m(x), y) < \tau\}}_{\text{expert } m \text{ agrees with label}}$$

The correspondence is: - **Hypothesis class**: Solomonoff sums over all computable functions; SCX sums over the finite set of M experts. - **Prior**: Solomonoff uses the universal prior $2^{-|p|}$; SCX uses state proportions ρ_s (experts are implicitly weighted by their state-conditioned reliability). - **Convergence**: Solomonoff converges to the truth with probability 1 for any computable environment (Hutter, 2005). SCX converges to a fixed point (Theorem SE-2) but may not converge to the truth (Theorem 3).

4.4 Key Limitations of the Analogy

1. **Hypothesis class restriction:** Solomonoff induction sums over **all** computable functions, making it a universal learner (incomputable but theoretically optimal). SCX sums over only M experts, which is a **severely restricted** hypothesis class. The SCX gatekeeper cannot represent a belief that none of the M experts captures.

$$S_{\infty}(x) \approx \text{Solomonoff predictor on a restricted hypothesis class}$$

2. **Computability:** Solomonoff induction is incomputable (the sum over all programs requires solving the halting problem). SCX's gatekeeper update is efficiently computable (polynomial in M , K_S , and N).
3. **No formal Occam prior:** SCX does not explicitly implement Occam's razor. Simpler state structures are preferred **implicitly** (k-means with few clusters is more stable; Proposition 6 favors reproducible partitions), but there is no formal simplicity penalty.
4. **No complete theory of convergence to truth:** Theorem 3 shows that SCX cannot always distinguish noise from difficulty, even in the infinite-data limit. Solomonoff induction does not have this limitation (any computable pattern is eventually learned).

4.5 Occam's Razor in SCX

While SCX does not explicitly implement Occam's razor, simpler state structures are implicitly preferred through:

- **k-means stability** (Proposition 6): Partitions with few, well-separated states have higher stability scores $S(\Phi, K_S)$. The stability diagnostic implicitly favors simpler state structures.
- **Cluster count selection**: Using the stability score to select K_S (choosing the smallest K_S with acceptable stability) implements a form of model selection that penalizes complexity.
- **Bootstrap consistency**: The bootstrap diagnostic (Proposition 6) detects when additional states are spurious (low ARI across resamples), discouraging overfitting in state discovery.

This implicit Occam bias is weaker than Solomonoff’s explicit prior $2^{-|p|}$ but serves a similar function: it prevents the gatekeeper from overfitting to noise in the expert consensus signals.

5. Comprehensive Comparison Table

5.1 Unified Comparison

Dimension	Bayesian AlphaZero	Active Optimization	Solomonoff Learning	Induction	SCX Self- Evolution
Core	Policy	Surrogate	Query	Universal	Lyapunov
Mech-	itera-	model +	strategy +	prior +	descent +
anism	tion +	acquisition	oracle	Bayesian	memory
	MCTS	function	labeling	update	bank +
	+				gatekeeper
	neural				
	net-				
	work				
Exploration	Donchik	Posterior	Uncertainty/discovery	Universal/discovery	Sensitivity
Strat-	noise +	variance	sampling	prior over all	function
egy	MCTS	(GP)		programs	$V(s) +$
	visit				consensus
	counts				uncertainty
Convergence	Monotonic	$\sqrt{T \cdot \gamma_T}$	$\tilde{O}(\theta \cdot d)$	Convergence	Finite-time
Guar-	policy	cumulative	label com-	to truth with	fixed point
antee	im-	regret; no	plexity;	prob 1 for	(Theorem
	prove-	exact	VC-based	computable	SE-2);
	ment;	termination		environments	Lyapunov
	opti-				descent
	mal for				(Theorem
	tabu-				SE-1)
	lar				
	case				

	Bayesian	Active	Solomonoff	SCX Self-	
Dimension	AlphaZero	Optimization	Learning	Induction	Evolution
Regret	$V^* - R_T =$	Label	Optimal but	$\Phi(q_t) -$	
Bound	$V^{\pi_t} \leq$ unknown(GP) (empir- ically fast)	$O(\sqrt{T \cdot \gamma_T})$ $O(\theta \cdot d \cdot \log(1/\varepsilon))$	complexity	incomputable	$\Phi_{\text{opt}} \leq$ $\Phi(q_0)e^{-\lambda t}$ (under contraction)
Computational	$O(T \cdot (n^3 +$	$O(n \cdot d)$	Incomputable	$O(M \cdot K_S \cdot$	
Cost	$d_{\text{net}})$ per it- eration	$dn))$ for GP per query		$N + K_S \cdot d_\phi)$ per iteration	
Key	Synthetic	Static	Point-wise	Universal	Real data;
Dif-	data;	black-box;	queries;	class but in-	unknown
fer-	known	point-wise;	single	computable;	f^* ;
ence	envi-	single	model; no	complete	state-level;
from	ron-	objective	state ag-	convergence	multi-
SCX	ment;		gregation		objective;
	stronger				finite
	conver-				compute
	gence				
Environment	Finite	Partially	Partially	Unknown	Unknown
	known	known (GP	known	but	physical law
	(game	prior)	(labeled	computable	(uncom-
	rules)		pool)		putable in
					general)

5.2 Detailed Metric Comparison

Metric	AlphaZero	BO	AL	SI	SCX
Per-iteration data cost	Free (synthetic)	$O(1)$ function evaluation	$O(b)$ labels per batch	None	$O(b)$ NEP validations
Scalability (X dimension)	High (neural nets)	Low (GP: $O(n^3)$)	Medium	None (incom-putable)	Medium (k-means: $O(Nd_\phi K_S)$)
Theoretical maturity	Empirical (except tabular)	Strong (GP regret)	Strong (dis-agreement coeff.)	Complete (incom-putable)	Partial (this work)
Applicability to physical sciences	Not designed (game domain)	High (materials, chemistry)	Medium (active labeling)	None (incom-putable)	High (designed for NEP)
Handles label noise	No (synthetic data)	No (GP assumes clean)	Partially (oracle assumed clean)	Yes (Bayesian)	Yes (primary design goal)

Metric	AlphaZero	BO	AL	SI	SCX
Uncertainty quantification	Value head variance	GP posterior	Model confidence	Full posterior	Multi-expert consensus
Memory of past data	Replay buffer (bounded)	All past evaluations ($O(n)$)	Labeled pool	All past data	Memory bank M_t
Formal convergence proof	Partial (tabular)	Yes (GP regret)	Yes (label complexity)	Yes (incom-putable)	Yes (SE-1, SE-2)

6. Synthesis: What Is and Is Not New

6.1 Aspects That Are Novel

1. **State-conditioned gatekeeper with Lyapunov convergence:**

The combination of a discrete state partition (from clustering) with a Lyapunov-stabilized update rule is absent from all four compared frameworks.

2. **Multi-expert consensus as an acquisition signal:** BO uses GP uncertainty; AL uses model confidence; SCX uses multi-expert consensus $C(x)$, which leverages M independent hypotheses rather than a single model's uncertainty.

3. **Memory-augmented self-correction with finite-time guarantee:** The monotonic memory bank + gatekeeper update loop with guaranteed finite-time termination (Theorem SE-2) has no direct analog in the compared frameworks.
4. **Completeness-incompleteness duality:** The simultaneous guarantee of termination (Theorem SE-2) and impossibility of self-certification (Claims SE-C1, SE-C2) is a structural feature not articulated in AL, BO, or AlphaZero.

6.2 Aspects That Are Not Novel

1. **Sequential experimental design:** The iterative selection-evaluation-update loop is the defining characteristic of all four compared frameworks. SCX does not invent a new paradigm here — it adapts existing paradigms to the NEP setting.
2. **Uncertainty-guided exploration:** Every framework in the comparison uses some form of uncertainty to guide exploration. SCX’s specific mechanism (multi-expert consensus) is novel, but the principle is not.
3. **Thompson sampling / posterior sampling:** SCX’s state-level value function $V(s)$ is a form of posterior-sampling-based acquisition, conceptually similar to Thompson sampling in BO.
4. **Occam’s razor via model selection:** The stability diagnostic (Proposition 6) implements a form of Occam’s razor that is conceptually similar to Bayesian model selection, though the implementation is different.

6.3 The SCX Synthesis

The SCX self-evolution framework occupies a **unique position** in the theory landscape:

- **Unlike AlphaZero:** It deals with real physical data (not synthetic), has unknown environment dynamics, and provides explicit convergence guarantees.
- **Unlike BO:** It operates at the state level (not point-wise), handles multiple experts (not a single surrogate), and addresses the epistemic limitation of self-certification.
- **Unlike AL:** It uses state-level certification (not point-wise queries), aggregates multiple independent experts (not a single model), and provides guarantees against label noise.
- **Unlike Solomonoff induction:** It is computationally tractable, operates over a restricted hypothesis class, and converges in finite time — at the cost of not guaranteeing convergence to truth.

The most accurate description is: **SCX is a restricted-domain, computationally tractable approximation of Solomonoff induction, operationalized through BO-inspired acquisition on a state space discovered by AL-like clustering, stabilized by AlphaZero-inspired monotonic improvement.**

References

1. Silver, D., et al. (2017). Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm. *arXiv:1712.01815*.

2. Silver, D., et al. (2018). A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419), 1140-1144.
3. Mockus, J. (1975). On Bayesian methods for seeking the extremum. *IFIP Technical Conference*.
4. Jones, D. R., et al. (1998). Efficient global optimization of expensive black-box functions. *Journal of Global Optimization*, 13(4), 455-492.
5. Srinivas, N., et al. (2010). Gaussian process optimization in the bandit setting: No regret and experimental design. *ICML 2010*.
6. Settles, B. (2009). Active learning literature survey. *Computer Sciences Technical Report 1648*, UW-Madison.
7. Balcan, M.-F., et al. (2009). Agnostic active learning. *Journal of Computer and System Sciences*, 75(1), 78-89.
8. Solomonoff, R. J. (1964). A formal theory of inductive inference. Part I. *Information and Control*, 7(1), 1-22.
9. Hutter, M. (2005). *Universal Artificial Intelligence*. Springer.
10. Li, M., & Vitanyi, P. (2008). *An Introduction to Kolmogorov Complexity and Its Applications* (3rd ed.). Springer.

End of 08_theory_connections.md